

Prozesssteckbrief

Reisekrankenversicherung

Feld	Beschreibung
Prozessname	Reisekrankenversicherung
Ziel	Automatisierte Bearbeitung eines Antrags auf Reisekrankenversicherung.
Auslöser	Ein Kunde stellt einen Antrag mit Reise-, Personen- und Versicherungsdaten.
Ergebnis	Der Antrag wird abgelehnt oder ein Vertrag wird gespeichert und die Unterlagen werden gedruckt/gesendet.
Technische Umsetzung	Camunda Cloud BPMN-Prozess mit Python-Zeebe-Worker und externen REST-APIs.

1. Zweck der Dokumentation

Dieser Prozesssteckbrief beschreibt fachlich und technisch den Prozess zur Reisekrankenversicherung. Er soll einen schnellen Überblick geben: Was macht der Prozess, welche Beteiligten gibt es, welche Systeme werden genutzt, welche Service Tasks existieren und welche Daten werden zwischen BPMN, Python Worker und APIs ausgetauscht.

2. Fachlicher Kurzüberblick

Ein Kunde stellt einen Antrag auf Reisekrankenversicherung. Danach werden die eingegebenen Daten geprüft. Je nach Ergebnis wird der Antrag entweder abgelehnt oder weiterverarbeitet. Anschließend wird geprüft, ob der Versicherungsnehmer bereits als Partner existiert. Falls nicht, wird ein neuer Partner angelegt. Zum Schluss wird der Versicherungsvertrag gespeichert und die Vertragsunterlagen werden gedruckt.

3. Beteiligte / Pools / Systeme

Beteiligter	Rolle im Prozess
Kunde	Stellt den Antrag.
Versicherung / Prozessanwendung	Hauptprozess: validiert Daten, steuert Entscheidungen, ruft Tasks auf und führt den Antrag bis zum Abschluss.
Sachbearbeitung	Bearbeitet manuelle Klärfälle, z. B. Adresse auswählen/anpassen oder Antrag prüfen.
travel-Insurance-API	Externes System zum Speichern des Versicherungsvertrags und zum Drucken der Vertragsunterlagen.
Camunda Cloud	Führt den BPMN-Prozess und DMN-Tabellen aus und übergibt Service Tasks an den Python Worker.
Python Worker	Registriert die Service Tasks und ruft die Internal APIs auf.

4. Grober Prozessablauf

1. Antrag wird gesendet.
2. Daten werden validiert (z. B. Reisebeginn, Reiseende und Kosten).
3. Falls Daten ungültig sind, wird der Antrag abgelehnt.
4. Die Reisekosten werden bei Bedarf in Euro umgerechnet.
5. Es wird geprüft, ob der Antrag aufgrund von Alter, Wohnort oder Personenanzahl abgelehnt werden muss.
6. Die Telefonnummer wird geprüft.
7. Es wird geprüft, ob eine Reisewarnung vorliegt. falls ja, wird der Antrag abgelehnt.
8. Versicherungsnehmer wird im Partnersystem gesucht.
9. Falls eine Partnernummer vorhanden ist, wird direkt nach dieser Nummer gesucht.
10. Falls keine Partnernummer vorhanden ist, wird über persönliche Daten gesucht.
11. Falls kein Partner gefunden wird, wird ein neuer Partner angelegt.
12. Falls Adresse nicht Übereinstimmt ist, kann die Sachbearbeitung eingreifen.
13. Selbstbehalt wird bestimmt.
14. Vertrag wird über die travel-Insurance-API gespeichert.
15. Vertragsunterlagen werden über die travel-Insurance-API gedruckt.
16. Prozess endet erfolgreich oder mit Ablehnung/Abbruch.

5.DMN Tabellen

Entscheidung	Beschreibung
Alter, Wohnort und Personen prüfen	Diese Entscheidung prüft, ob die Antragstellerin die grundlegenden Voraussetzungen erfüllt. Dabei werden Alter, Herkunftsland und Anzahl der versicherten Personen kontrolliert. Wenn eine Bedingung nicht erfüllt ist, wird ein passender Grund ausgegeben.
Selbstbehalt bestimmen	Diese Entscheidung berechnet den Selbstbehalt der Reisekrankenversicherung. Der Betrag hängt vom Kundenstatus, der Reisedauer und dem Reiseziel ab. Bei langen Reisen außerhalb der EU wird der Selbstbehalt anhand der Reisekosten berechnet.

6. Wichtige Entscheidungen / Gateway

Entscheidung	Mögliche Ausgänge	Bedeutung
Daten gültig?	ja / nein	Nur gültige Anträge werden weiterverarbeitet. Ungültige Daten führen zu Rückfrage oder Ablehnung.
Währung ist EUR?	ja / nein	Prüft, ob die Reisekosten in Euro vorliegen oder umgerechnet werden müssen.
Antrag zulässig?	ja / nein	Prüft fachliche Voraussetzungen, z. B. ob der Antrag grundsätzlich bearbeitet werden darf.
Telefonnummer gültig?	ja / nein	Prüft, ob alle relevanten Teilnehmerdaten vorhanden und plausibel sind.
Reisewarnung vorhanden?	ja / nein	Bei Reisewarnung erfolgt eine Ablehnung.
Partnernummer angegeben?	ja / nein	Steuert, ob direkt per Partnernummer oder über Personendaten gesucht wird.
Partner per Daten gefunden?	ja / nein	Entscheidet, ob ein vorhandener Partner genutzt oder ein neuer Partner angelegt wird.
Partner mit Partnernummer gefunden?	ja / nein	Wenn der Partner gefunden wurde, wird die Adresse geprüft; wenn nicht, sucht die Sachbearbeitung den Partner manuell.
Partner manuell gefunden?	ja / nein	Prüft, ob durch die manuelle Suche ein passender Partner gefunden wurde.
Adresse stimmt überein?	ja / nein	Bei abweichender Adresse kann die Sachbearbeitung die Adresse auswählen oder korrigieren.

7. Service Tasks und technische Zuordnung

BPMN Task Type	Python-Funktion	Datei	Aufgabe
Daten validieren	validate_data	tasks.py	Prüft Reisedaten: Start vor Ende, Start in der Zukunft, Kosten positiv.
VN im Partnersystem suchen	search_vn	tasks.py + partner_api.py	Prüft ob die Partnernummer gegeben ist oder nicht
Partner im System suchen	search_partner	tasks.py + partner_api.py	Sucht Partner über persönliche Daten und Adresse.
Neukunde im System anlegen	create_partner_task	tasks.py + partner_api.py	Legt einen neuen Partner im Partnersystem an.
Partner als Neukunde im System anlegen	create_partner_task	tasks.py + partner_api.py	Legt einen neuen Partner im Partnersystem an.
Vertrag Speichern	vertrag_speichern	tasks.py + insurance_api.py	Der Vertrag wird im Versicherungssystem gespeichert.
Vertragsunterlagen drucken	unterlagen_senden	tasks.py + insurance_api.py	Startet Druckauftrag für Vertragsunterlagen.

8. User Tasks

User Task	Zweck
Sachbearbeitung Adresse auswählen/anpassen	Bei Adressabweichung wird entschieden, welche Adresse verwendet werden soll.
Sachbearbeitung: Partner manuell suchen	Wenn kein Partner über die Partnernummer gefunden wurde, sucht die Sachbearbeitung den Partner manuell im System.

9. Wichtige Prozessvariablen

Variable	Bedeutung
travellInsurance	Hauptobjekt mit Antrag, Reisedaten, Versicherungsnehmer und versicherten Personen.
travellInsurance.travelData	Reisedaten wie Start, Ende, Kosten und Währung.
travellInsurance.policyHolder	Versicherungsnehmer mit Name, Geburtstag, Adresse, Mail, Telefon und ggf. Partnernummer.
travellInsurance.insuredPartners	Liste der versicherten Personen.
datenGueltig	Ergebnis der Datenvalidierung.
telefonnummerGueltig	Ergebnis der Telefonnummernprüfung.
partnernummerAngegeben	Gibt an, ob im Antrag eine Partnernummer vorhanden ist.
partnerPerSucheGefunden	Ergebnis der Partnersuche ohne Partnernummer.
partnerMitPartnernummerGefunden	Ergebnis der Partnersuche mit angegebener Partnernummer.
isChecked	Gibt an, ob die Sachbearbeitung den Partner manuell geprüft hat.
partnernummer	Partner-ID aus dem Partnersystem.
partnerAusApi	Partnerdaten aus der Travel-insurance-API.
adresseStimmtUeberein	Ergebnis des Adressvergleichs.
versicherungsnummer	Nummer der gespeicherten Versicherungspolice.
apiFehler	Kennzeichen, dass ein technischer Fehler bei API-Aufrufen passiert ist.
unterlagenGesendet	Ergebnis des Druck-/Sendeauftrags.

10. Python-Dateien und Verbindung

Datei	Aufgabe	Verbindung
config.py	Speichert API-URL, API-Zugangsdaten und Camunda-Zugangsdaten.	Wird von main.py, partner_api.py und insurance_api.py importiert.
main.py	Startet den Worker, baut die Verbindung zu Camunda Cloud auf und registriert die Tasks.	Ruft register_tasks(worker) aus tasks.py auf.
tasks.py	Enthält die Service Tasks für Camunda.	Nutzt partner_api.py, insurance_api.py und utils.py.
partner_api.py	Kapselt alle API-Aufrufe zum Partnersystem.	Nutzt config.py für URL und Login. Wird von tasks.py genutzt.
insurance_api.py	Kapselt Speichern der Versicherung und Drucken der Unterlagen.	Nutzt config.py. Wird von tasks.py genutzt.
utils.py	Enthält Hilfsfunktionen, z. B. Normalisierung und Adressvergleich.	Wird von tasks.py verwendet.

11. Technische Architektur

Die Verbindung zwischen BPMN und Python funktioniert über Camunda Cloud und einen Zeebe Worker. Der BPMN-Prozess erreicht einen Service Task. Camunda sucht dann einen Worker, der den passenden task_type registriert hat. Der Python Worker führt die Funktion aus und gibt neue Prozessvariablen an Camunda zurück.

Schritt	Technischer Ablauf
1	main.py baut mit Camunda-Zugangsdaten einen Cloud Channel auf.
2	Ein ZeebeWorker wird erstellt.
3	tasks.py registriert alle Service Tasks mit @worker.task(task_type=...).
4	Camunda übergibt Jobs an den Worker.
5	Der Worker liest Prozessvariablen, ruft ggf. externe APIs auf und gibt Ergebnisse zurück.
6	Gateways im BPMN entscheiden anhand dieser Variablen über den weiteren Ablauf.

12. Intern API-Aufrufe

Funktion	HTTP-Aufruf	Zweck
search_partner_by_number	GET /partner/{partnernummer}	Partner direkt per Partnernummer suchen.
search_partner_by_data	POST /partner/search	Partner über Name, Geburtsdatum, Kontakt und Adresse suchen.
create_partner	POST /partner	Neuen Partner anlegen.
save_insurance_policy	POST /insurance-policy	Versicherungsvertrag speichern und Versicherungsnummer erhalten.
print_documents	POST /document/print-job/insurance-policy/{policy_id}	Druckauftrag für Vertragsunterlagen starten.

13. Extern API-Aufrufe

API-Aufruf	Methode	Zweck
Währungsumrechnung API	GET	Reisekosten in EUR umrechnen, wenn Währung nicht EUR ist.
Telefonnummer prüfen	GET	Prüfen, ob die Telefonnummer gültig ist.
Reisewarnung prüfen	GET	Prüfen, ob für das Reiseziel eine Reisewarnung existiert.
Bestätigungsmail versenden	POST / Mail-Service	Kunde per E-Mail über erfolgreichen Abschluss informieren